

Linguaggi Formali e Compilatori

Argomento 05: Parsing Ascendente

Prof. Giuseppe Psaila

Laurea Magistrale in Ingegneria Informatica
Università di Bergamo

- Ricostruisce le derivazioni **Canoniche Destre**
- Denominata **LR(0)**
Left-to-Right
Right-most
Look-ahead 0
- Cioè senza look-ahead, si consuma il simbolo e si fa qualche cosa
(ma senza look-ahead risulta limitata, quindi poi lo introdurremo)

Parti Destre e Automi

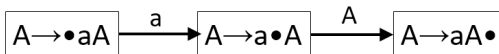
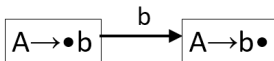
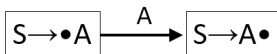
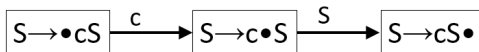
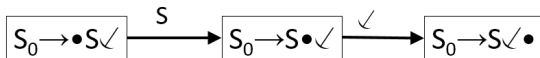
- La parte destra di una regola di produzione $A \rightarrow \alpha$ è una stringa basata sull'alfabeto $(V \cup \Sigma)$
- Un semplice automa a stati finiti con alfabeto $(V \cup \Sigma)$ è in grado di riconoscerla
- Possiamo immaginare di avere tanti micro-automi, uno per ogni regola

Parti Destre e Automi

Consideriamo la grammatica seguente, con $\Sigma = \{a, b, c\}$ e $V = \{S, A\}$ (S_0 è il **Super-Assioma**)

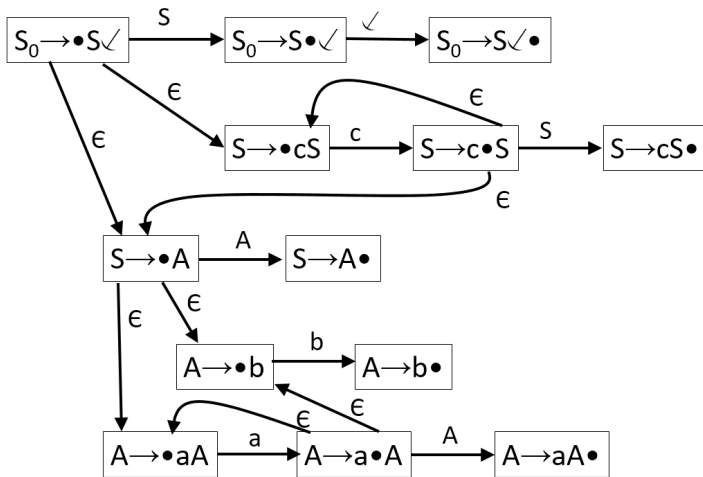
- $S_0 \rightarrow S$ ✓
 $S \rightarrow A$
 $S \rightarrow c S$
 $A \rightarrow a A$
 $A \rightarrow b$

Parti Destre e Automi



- Il simbolo **•** indica la **testina di lettura**, quindi quando un simbolo viene consumato, si sposta a **destra**
- Ma che cosa vuol dire
Consumare un Non-Terminale?
- Vuol dire **Aver Riconosciuto il Sotto-Albero in Esso Radicato**
- IDEA: mettiamo delle ϵ -mosse di raccordo, ma così facendo l'automa diventa non-deterministico
- Inoltre, occorre associare una pila per gestire gli annidamenti e i punti di decisione

Parti Destre e Automi



- Quando una regola di produzione viene riconosciuta
- sul dispositivo di ingresso viene messo il simbolo Non-Terminale padre della produzione
- quindi si deve tornare indietro allo stato dal quale è stata fatta la mossa ϵ .
- Come fare a eliminare il non-determinismo?
Raccogliendo in un unico stato tutte le regole che espandono un **non-terminale marcato** da •

Costruzione Automa LR(0)

- Data la grammatica $G(V, \Sigma, P, S_0)$
(con $S_0 \rightarrow S \quad \swarrow \in P$ e S_0 **Non Ricorsivo**)
- Uno stato s è un **Insieme di Candidate**

Costruzione Automa LR(0): Candidate

- Una candidata c ha la forma

$$N \rightarrow \alpha \bullet \beta$$

con $\alpha, \beta \in (V \cup \Sigma)^*$

e $N \rightarrow \alpha\beta \in P$

Costruzione Automa LR(0): Candidate di Spostamento/Riduzione

- Candidata di **SPOSTAMENTO**:

Una candidata $N \rightarrow \alpha \bullet \beta$ è detta di **Spostamento** se $|\beta| > 0$

- Candidata di **RIDUZIONE**:

Una candidata $N \rightarrow \alpha \bullet \beta$ è detta di **Riduzione** se $|\beta| = 0$

Costruzione Automa LR(0): Candidate Core/di Completamento

- Candidata **CORE**:

Una candidata $N \rightarrow \alpha \bullet \beta$ è detta **CORE** se $|\alpha| \neq 0$ o se è $S_0 \rightarrow \bullet S$ ✓

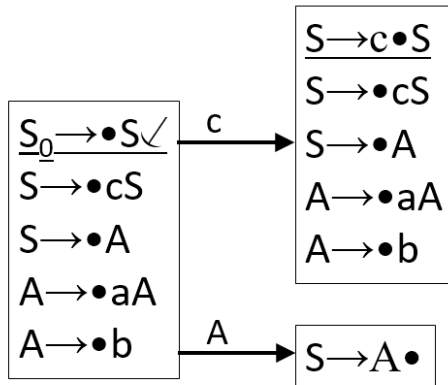
- Candidata di **COMPLETAMENTO**:

Una candidata $N \rightarrow \alpha \bullet \beta$ è detta di **COMPLETAMENTO** se $|\alpha| = 0$ e se vi è una candidata $\bar{c} = M \rightarrow \bar{\alpha} \bullet N\bar{\beta}$ nello stesso stato s

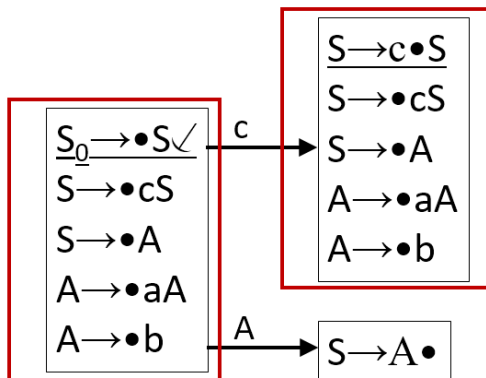
Costruzione Automa LR(0): Mosse Uscenti

- Dati due stati s_1 e s_2 , nell'automa esiste una transizione $s_1 \xrightarrow{A} s_2$ (con $A \in (V \cup \Sigma)$) se
- in s_1 vi è un insieme di candidate $m(A) = \{c_A_1, c_A_2, \dots, c_A_n\} \neq \emptyset$, con $c_A_i : N_i \rightarrow \alpha_i \bullet A \beta_i$
- e in s_2 vi è un insieme $\overline{m}(A) = \{\overline{c_A_1}, \overline{c_A_2}, \dots, \overline{c_A_n}\}$ tale che per ogni candidata $c_A_i \in m(A)$, esiste la corrispondente candidata $\overline{c_A_i}$ tale che $\overline{c_A_i} : N_i \rightarrow \alpha_i A \bullet \beta_i$

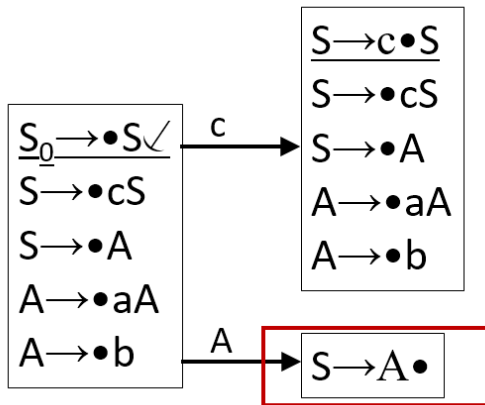
Alcuni Stati



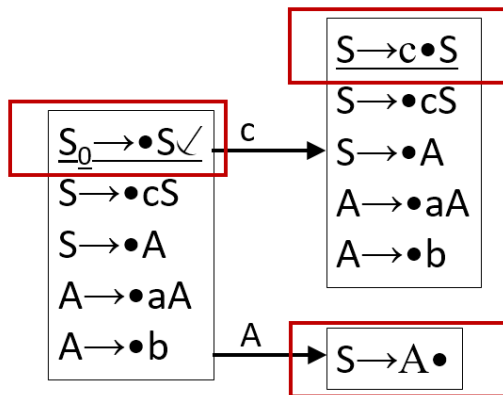
Candidate di Spostamento



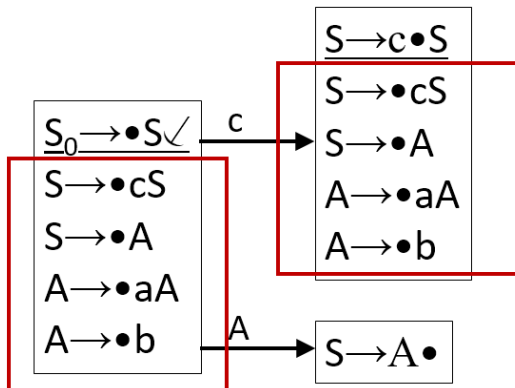
Candidate di Riduzione



Candidate Core



Candidate di Completamento



Procedimento Costruttivo

- **Passo 1**

Lo stato iniziale I_1 contiene la candidata core

$$S_0 \rightarrow \bullet S \quad \checkmark$$

- **Passo 2**

Per ogni stato s , per ogni candidata $M \rightarrow \alpha \bullet N \beta$ con $N \in V$

si aggiungono in s le candidate di **completamento**

$N \rightarrow \bullet \alpha_i$, per ogni regola di produzione $N \rightarrow \alpha_i \in P$.

Si continua ad aggiungere candidate di completamento fino a che tutti i simboli non-terminali marcati con $\bullet$ sono stati completati

• Passo 3

Per ogni stato s , si determinano le mosse uscenti.

Per ogni gruppo di candidate $m(A) = \{c_A_i\}$, con

$c_A_i : N_i \rightarrow \alpha_i \bullet A\beta_i$ (e con $A \in (V \cup \Sigma)$),

cioè le candidate con lo stesso simbolo marcato da $\bullet$,

si crea (se non è già presente) uno stato $\bar{s}$ le cui

candidate core sono esattamente $\bar{m}(A) = \{\overline{c_A_i}\}$

(con $|m(A)| = |\bar{m}(A)|$),

tale che $\overline{c_A_i} : N \rightarrow \alpha_i A \bullet \beta_i$ deriva da

$c_A_i : N_i \rightarrow \alpha_i \bullet A\beta_i$.

Si aggiunge la transizione $s \xrightarrow{A} \bar{s}$.

• Per ogni nuovo stato s , si ripete dal **Passo 2**.

Conflitti

- Si dice che uno stato s presenta un **Conflitto SPOSTAMENTO/RIDUZIONE** se contiene sia una candidata di spostamento che una candidata di riduzione
- Si dice che uno stato s presenta un **Conflitto RIDUZIONE/RIDUZIONE** se contiene due diverse candidate di riduzione

Automa LR(0)

- Uno stato s è detto **LR(0)** se non contiene né conflitti SPOSTAMENTO/RIDUZIONE né conflitti RIDUZIONE/RIDUZIONE.
- Un Automa è detto **LR(0)** se tutti i suoi stati sono LR(0).

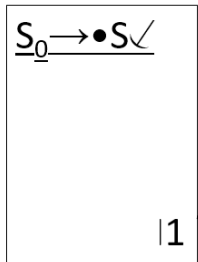
Conseguentemente, anche la grammatica è detta LR(0).

Automa LR(0) per la grammatica

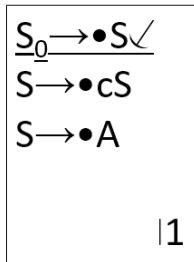
$\Sigma = \{a, b, c\}$ e $V = \{S, A\}$ (S_0 è il **Super-Assioma**)

- $S_0 \rightarrow S$ ✓
 $S \rightarrow A$
 $S \rightarrow c S$
 $A \rightarrow a A$
 $A \rightarrow b$

Parsing Ascendente



Parsing Ascendente



Parsing Ascendente

$S_0 \rightarrow \bullet S \checkmark$

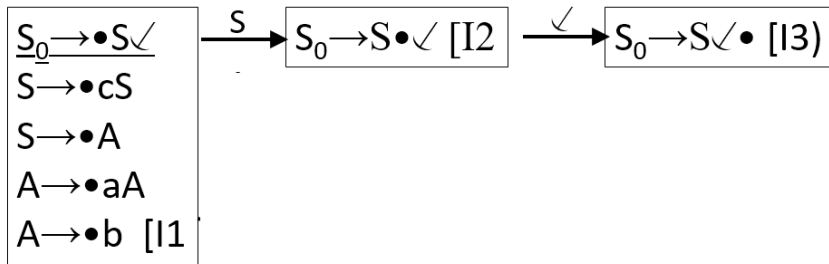
$S \rightarrow \bullet cS$

$S \rightarrow \bullet A$

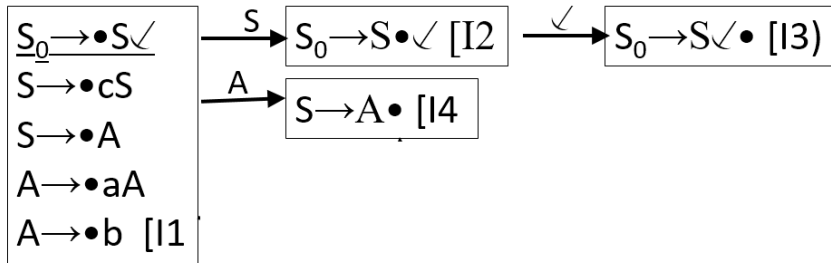
$A \rightarrow \bullet aA$

$A \rightarrow \bullet b$ [11]

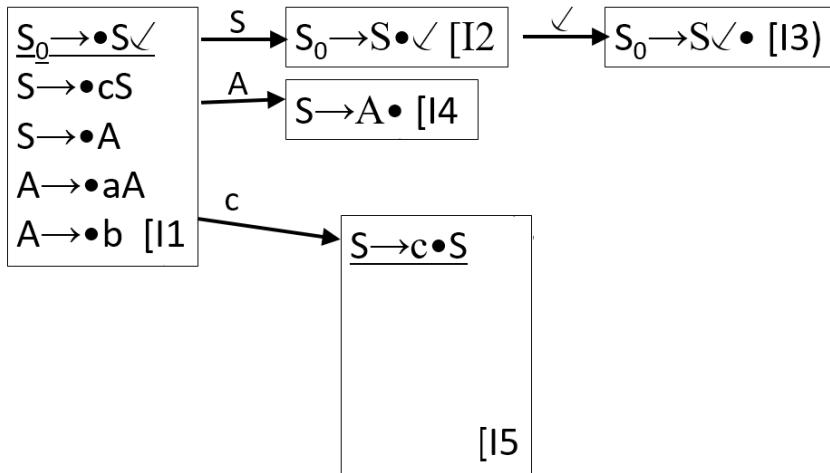
Parsing Ascendente



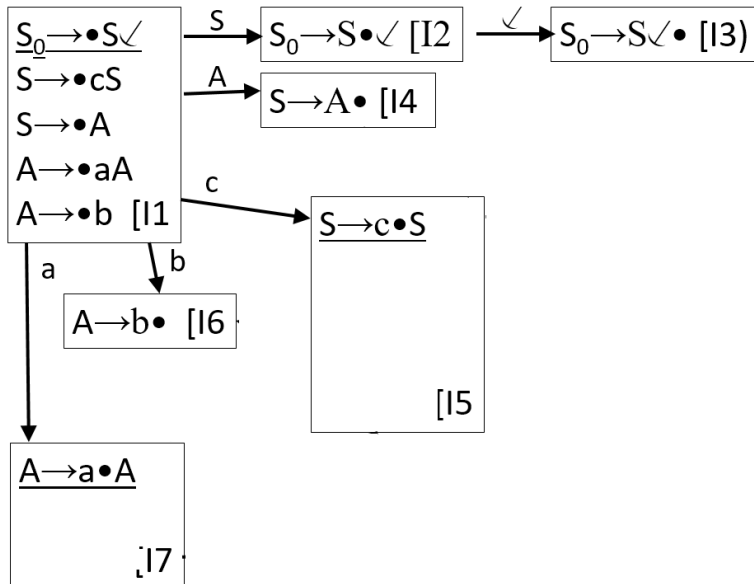
Parsing Ascendente



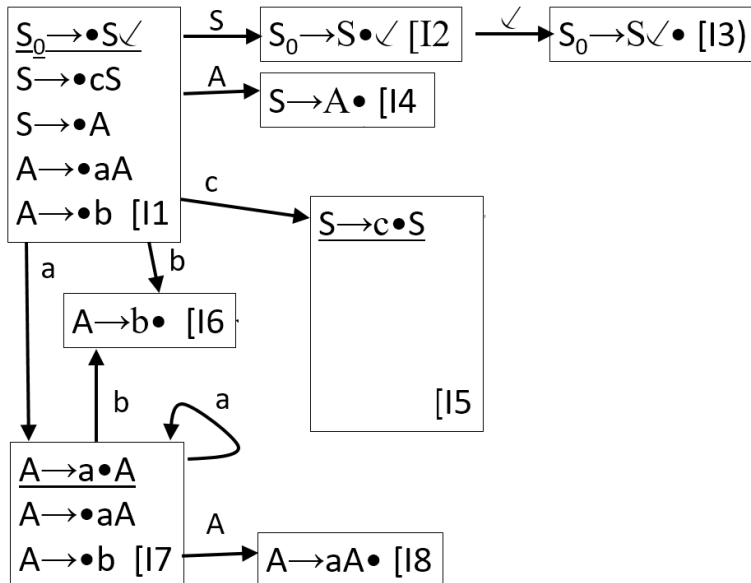
Parsing Ascendente



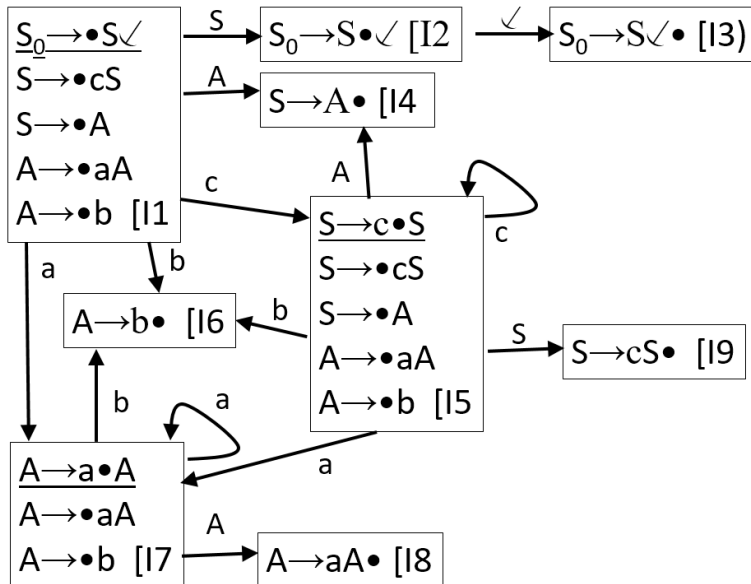
Parsing Ascendente



Parsing Ascendente



Parsing Ascendente



Funzionamento dell'Automa

- **Passo 1**

Inizializzare la pila con lo stato iniziale I_1

- **Passo 2**

Con uno stato s in cima alla pila,
consumare il simbolo c dal dispositivo di ingresso e
metterlo in cima alla pila.

- **Passo 3**

Con una coppia (s, c) in cima alla pila, dove s è uno
stato e $c \in (V \cup \Sigma)$,
se la transizione $s \xrightarrow{c} s'$ NON è definita, segnalare
errore,
altrimenti mettere s' in cima alla pila

Funzionamento dell'Automa

- **Passo 4**

Se lo stato s in cima alla pila può ridurre la regola $N \rightarrow \alpha$ (dalla candidata $N \rightarrow \alpha \bullet$)

rimuovere dalla pila $n = |\alpha|$ coppie (c, s)

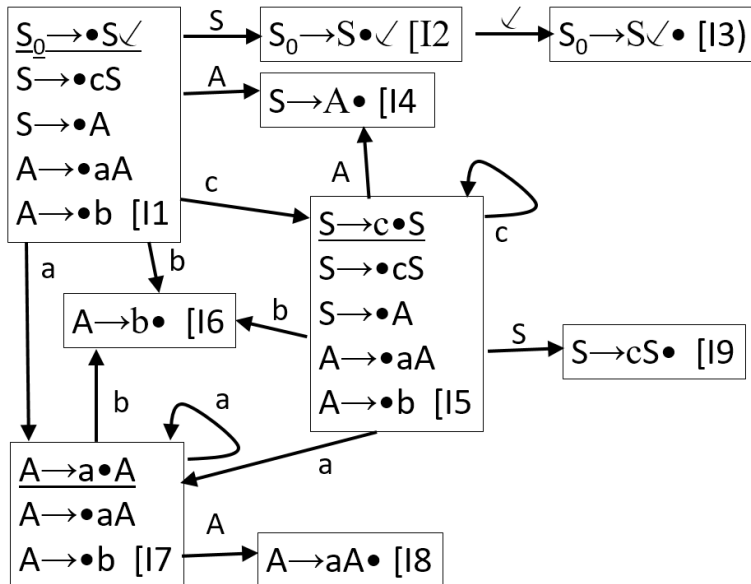
e impilare N (parte sinistra della regola ridotta).

- Se la pila contiene solo $[I_1, S_0]$, la stringa è stata riconosciuta
altrimenti ripetere dal **Passo 2**

Esempio: stringa *cab* ✓

Stringa	Pila	Azione
<i>cab</i> ✓	I_1	

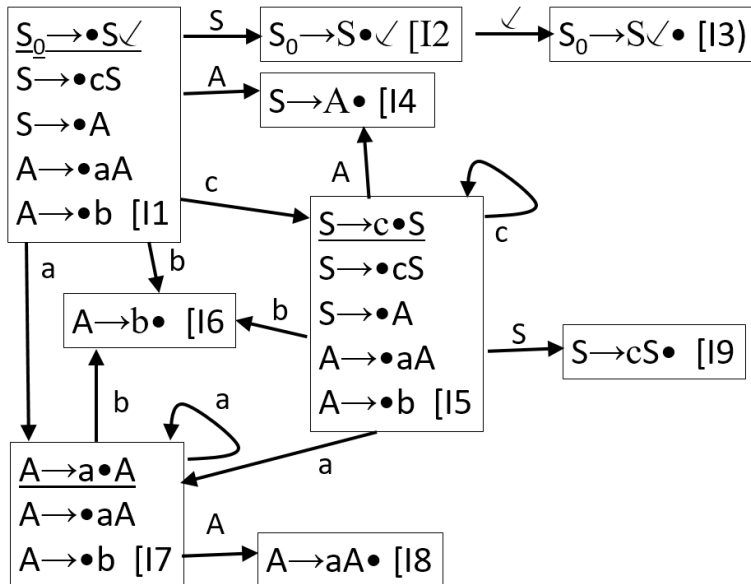
Parsing Ascendente



Esempio: stringa cab ✓

Stringa	Pila	Azione
cab ✓	l_1	
ab ✓	l_1cl_5	

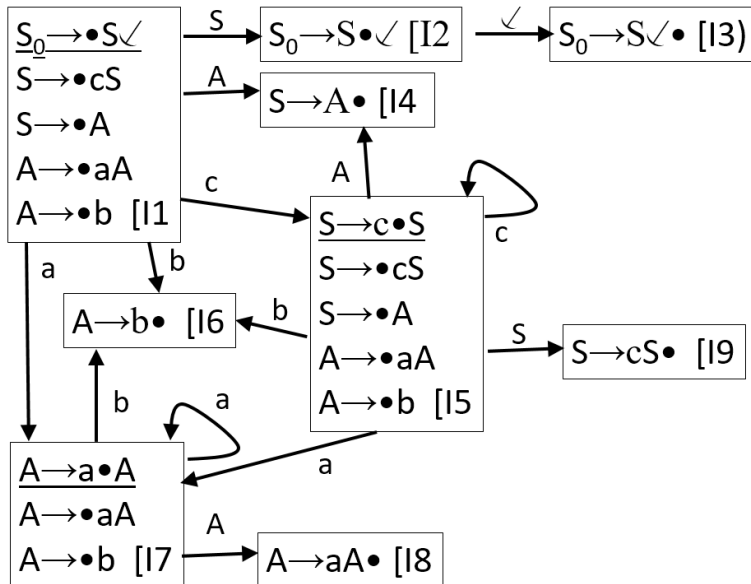
Parsing Ascendente



Esempio: stringa cab ✓

Stringa	Pila	Azione
cab ✓	l_1	
ab ✓	l_1cl_5	
b ✓	$l_1cl_5al_7$	

Parsing Ascendente



Esempio: stringa cab ✓

Stringa	Pila	Azione
cab ✓	l_1	
ab ✓	l_1cl_5	
b ✓	$l_1cl_5al_7$	
✓	$l_1cl_5al_7bl_6$	Riduzione $A \rightarrow b$

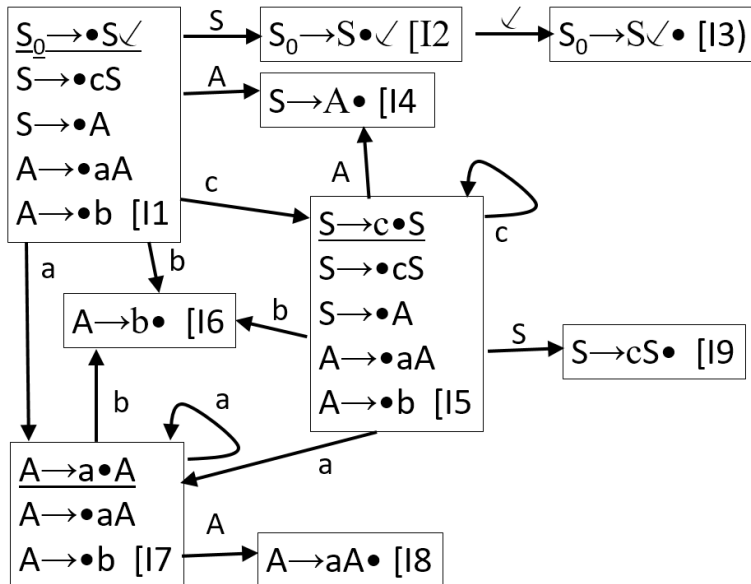
Parsing Ascendente



Esempio: stringa cab ✓

Stringa	Pila	Azione
cab ✓	l_1	
ab ✓	l_1cl_5	
b ✓	$l_1cl_5al_7$	
✓	$l_1cl_5al_7bl_6$	Riduzione $A \rightarrow b$
✓	$l_1cl_5al_7A$	

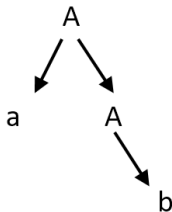
Parsing Ascendente



Esempio: stringa cab ✓

Stringa	Pila	Azione
cab ✓	I_1	
ab ✓	$I_1 c I_5$	
b ✓	$I_1 c I_5 a I_7$	
✓	$I_1 c I_5 a I_7 b I_6$	Riduzione $A \rightarrow b$
✓	$I_1 c I_5 a I_7 A I_8$	Riduzione $A \rightarrow aA$

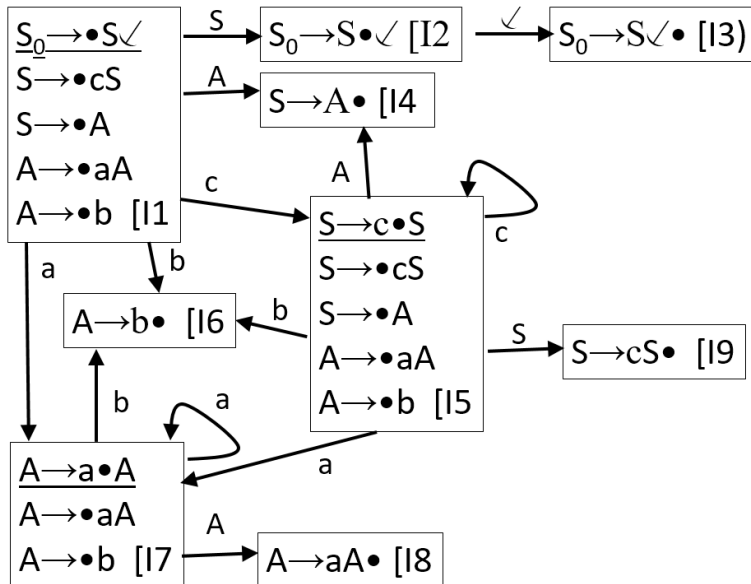
Parsing Ascendente



Esempio: stringa cab ✓

Stringa	Pila	Azione
cab ✓	l_1	
ab ✓	l_1cl_5	
b ✓	$l_1cl_5al_7$	
✓	$l_1cl_5al_7bl_6$	Riduzione $A \rightarrow b$
✓	$l_1cl_5al_7Al_8$	Riduzione $A \rightarrow aA$
✓	l_1cl_5A	

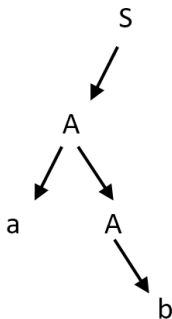
Parsing Ascendente



Esempio: stringa cab ✓

Stringa	Pila	Azione
cab ✓	l_1	
ab ✓	l_1cl_5	
b ✓	$l_1cl_5al_7$	
✓	$l_1cl_5al_7bl_6$	Riduzione $A \rightarrow b$
✓	$l_1cl_5al_7Al_8$	Riduzione $A \rightarrow aA$
✓	$l_1cl_5Al_4$	Riduzione $S \rightarrow A$

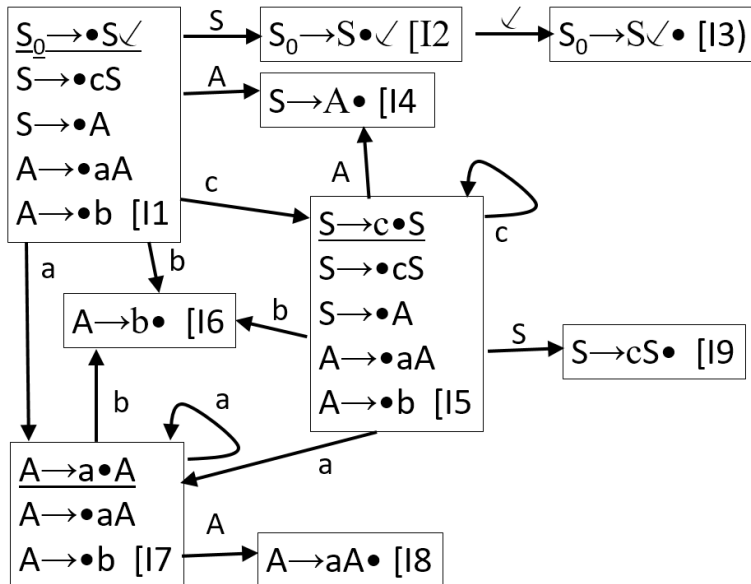
Parsing Ascendente



Esempio: stringa cab ✓

Stringa	Pila	Azione
cab ✓	l_1	
ab ✓	l_1cl_5	
b ✓	$l_1cl_5al_7$	
✓	$l_1cl_5al_7bl_6$	Riduzione $A \rightarrow b$
✓	$l_1cl_5al_7Al_8$	Riduzione $A \rightarrow aA$
✓	$l_1cl_5Al_4$	Riduzione $S \rightarrow A$
✓	l_1cl_5S	

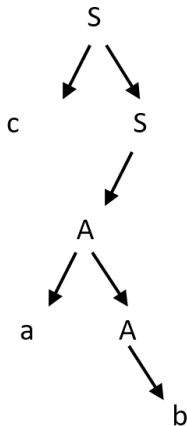
Parsing Ascendente



Esempio: stringa cab ✓

Stringa	Pila	Azione
cab ✓	l_1	
ab ✓	l_1cl_5	
b ✓	$l_1cl_5al_7$	
✓	$l_1cl_5al_7bl_6$	Riduzione $A \rightarrow b$
✓	$l_1cl_5al_7Al_8$	Riduzione $A \rightarrow aA$
✓	$l_1cl_5Al_4$	Riduzione $S \rightarrow A$
✓	$l_1cl_5Sl_9$	Riduzione $S \rightarrow cS$

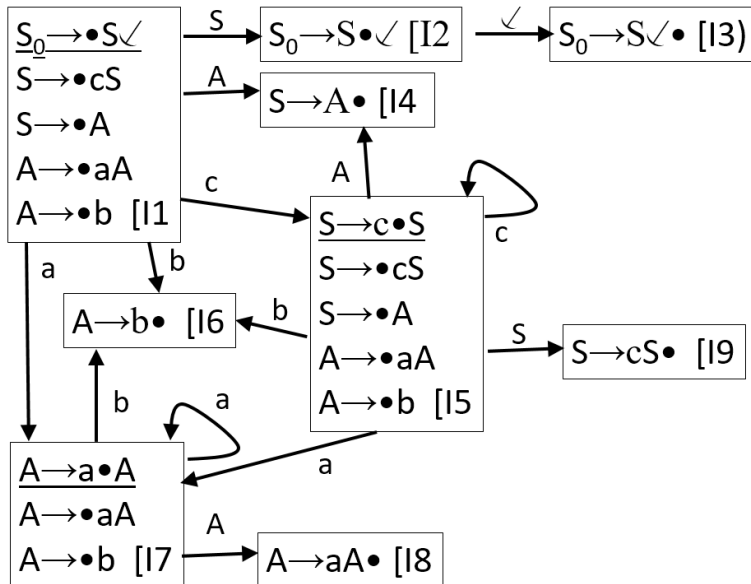
Parsing Ascendente



Esempio: stringa cab ✓

Stringa	Pila	Azione
cab ✓	l_1	
ab ✓	l_1cl_5	
b ✓	$l_1cl_5al_7$	
✓	$l_1cl_5al_7bl_6$	Riduzione $A \rightarrow b$
✓	$l_1cl_5al_7Al_8$	Riduzione $A \rightarrow aA$
✓	$l_1cl_5Al_4$	Riduzione $S \rightarrow A$
✓	$l_1cl_5Sl_9$	Riduzione $S \rightarrow cS$
✓	l_1S	

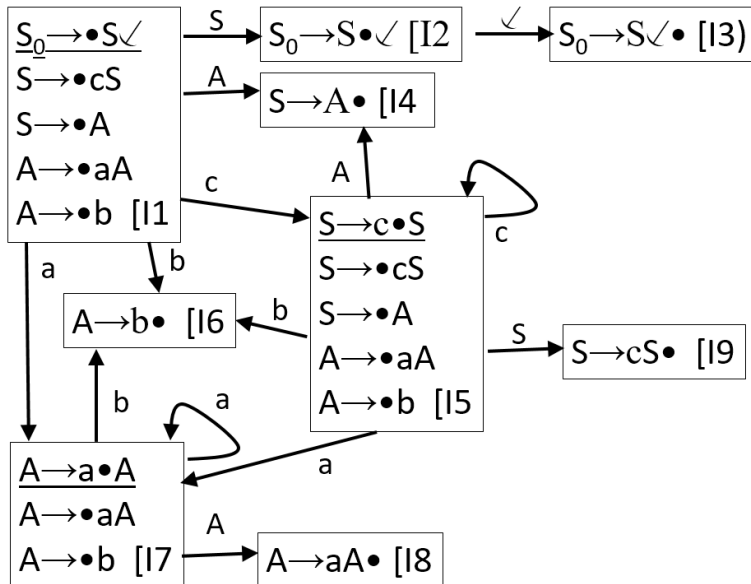
Parsing Ascendente



Esempio: stringa cab ✓

Stringa	Pila	Azione
cab ✓	I_1	
ab ✓	$I_1 c I_5$	
b ✓	$I_1 c I_5 a I_7$	
✓	$I_1 c I_5 a I_7 b I_6$	Riduzione $A \rightarrow b$
✓	$I_1 c I_5 a I_7 A I_8$	Riduzione $A \rightarrow aA$
✓	$I_1 c I_5 A I_4$	Riduzione $S \rightarrow A$
✓	$I_1 c I_5 S I_9$	Riduzione $S \rightarrow cS$
✓	$I_1 S I_2$	

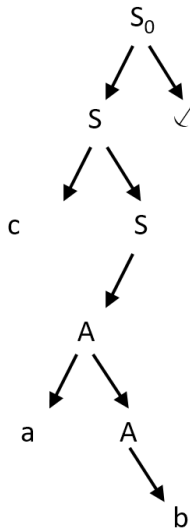
Parsing Ascendente



Esempio: stringa cab ✓

Stringa	Pila	Azione
cab ✓	l_1	
ab ✓	l_1cl_5	
b ✓	$l_1cl_5al_7$	
✓	$l_1cl_5al_7bl_6$	Riduzione $A \rightarrow b$
✓	$l_1cl_5al_7Al_8$	Riduzione $A \rightarrow aA$
✓	$l_1cl_5Al_4$	Riduzione $S \rightarrow A$
✓	$l_1cl_5Sl_9$	Riduzione $S \rightarrow cS$
✓	l_1Sl_2	
	l_1Sl_2 ✓ l_3	Riduzione $S_0 \rightarrow S$ ✓

Parsing Ascendente



Esempio: stringa cab ✓

Stringa	Pila	Azione
cab ✓	l_1	
ab ✓	l_1cl_5	
b ✓	$l_1cl_5al_7$	
✓	$l_1cl_5al_7bl_6$	Riduzione $A \rightarrow b$
✓	$l_1cl_5al_7Al_8$	Riduzione $A \rightarrow aA$
✓	$l_1cl_5Al_4$	Riduzione $S \rightarrow A$
✓	$l_1cl_5Sl_9$	Riduzione $S \rightarrow cS$
✓	l_1Sl_2	
	l_1Sl_2 ✓ l_3	Riduzione $S_0 \rightarrow S$ ✓
	l_1S_0	OK

Esercizio

- $\Sigma = \{a, b, x, y, \swarrow\}$, $V = \{S_0, S, A, B\}$
- $S_0 \rightarrow S \swarrow$
 $S \rightarrow A B y$
 $A \rightarrow a A$
 $A \rightarrow x$
 $B \rightarrow B b$
 $B \rightarrow y$
- È LR(0)?

Insiemi di Prospezione LALR(1)

- Come potenziare la tecnica LR(0)?
- introducendo gli **Insiemi di Prospezione** (**Look-Ahead Set**) per le candidate di riduzione che creano un conflitto

Insiemi di Prospezione LALR(1)

- Dato l'automa di tipo LR(0), ma **NON LR(0)**,
- negli stati non LR(0) si associano alle **candidate di riduzione** gli insiemi di prospezione $LA(s, N \rightarrow \alpha\bullet)$ o, se lo stato s è sottinteso, $LA(N \rightarrow \alpha\bullet)$.
- $LA(s, N \rightarrow \alpha\bullet) = \{a \in \Sigma \mid \exists S_o \xrightarrow{*} \gamma N a z \rightarrow \gamma \alpha a z \text{ AND } \delta^*(I_1, \gamma \alpha) = s\}$
derivazione destra

Condizioni LALR(1)

- Per ogni stato s contenente $LA(s, N \rightarrow \alpha\bullet)$
- Per ogni coppia di candidate
 $N_1 \rightarrow \alpha_1\bullet$ e $N_2 \rightarrow \alpha_2\bullet$
deve essere $LA(s, N_1 \rightarrow \alpha_1\bullet) \cap LA(s, N_2 \rightarrow \alpha_2\bullet) = \emptyset$
- Per ogni candidata di riduzione $N \rightarrow \alpha\bullet$
deve essere
 $LA(s, N \rightarrow \alpha\bullet) \cap \{b \in \Sigma \mid \delta(s, b) \text{ è definita}\} = \emptyset$
- Se queste due condizioni sono verificate, lo stato s viene detto **ADEGUATO**.

Grammatica LALR(1)

- La grammatica G è LALR(1)
se **Tutti gli Stati dell'Automa sono ADEGUATI**
(uno stato LR(0) è, di per sé, adeguato).

Metodo Operativo Calcolo LA

In uno stato s , data una candidata $c : N \rightarrow \alpha \bullet \beta$
vogliamo calcolare $Seg_s(N \rightarrow \alpha \bullet \beta)$

- Se $|\alpha| > 0$, candidata core, i seguiti arrivano da stati esterni:

$$Seg_s(N \rightarrow \alpha \bullet \beta) = \cup_{s_j} Seg_{s_j}(N \rightarrow \bullet \alpha \beta)$$

per ogni stato s_j per cui $\delta^*(s_j, \alpha) = s$

- Se $|\alpha| = 0$, candidata di completamento, i seguiti arrivano da altre candidate nello stesso stato s :

$$Seg_s(N \rightarrow \bullet \beta) = \cup_i Seg_s(N, c_i)$$

per ogni candata $c_i : M_i \rightarrow \alpha_i \bullet N \beta_i$

In particolare:

- $Seg_s(N, c_i) = Ini(\beta_i)$ se $\neg Annullabile(\beta_i)$
- $Seg_s(N, c_i) = Ini(\beta_i) \cup Seg_s(M_i \rightarrow \alpha_i \bullet N\beta_i)$
se $Annullabile(\beta_i)$

In uno stato s NON LR(0), data una candidata di riduzione $N \rightarrow \alpha\bullet$,

si calcola

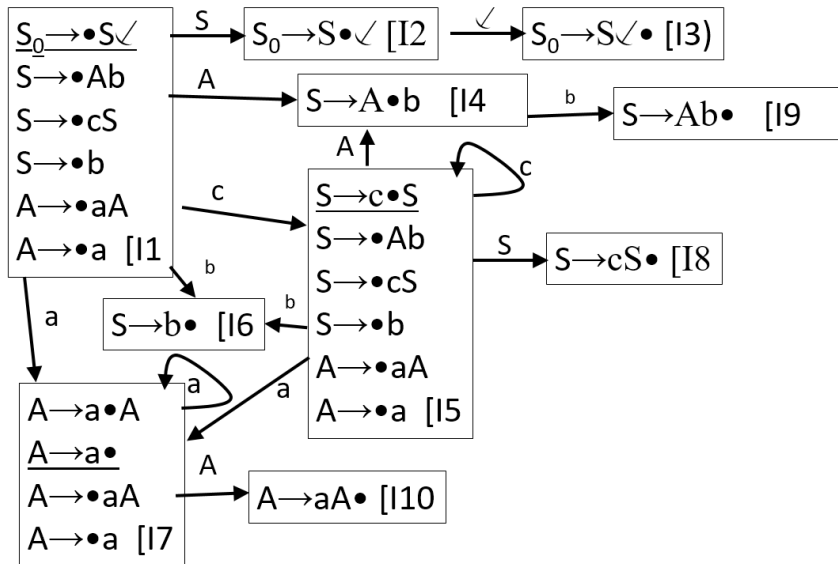
$$LA(N \rightarrow \alpha\bullet) = Seg_s(N \rightarrow \alpha\bullet)$$

La grammatica è LR(0)?

$\Sigma = \{a, b, c\}$, $V = \{S, A, \}$

- $S_0 \rightarrow S$ ✓
- $S \rightarrow Ab$
- $S \rightarrow cS$
- $S \rightarrow b$
- $A \rightarrow aA$
- $A \rightarrow a$

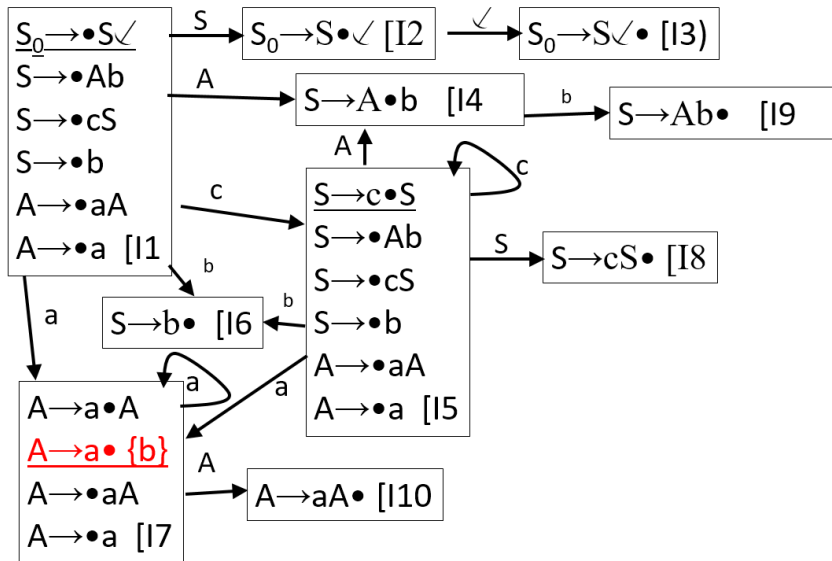
Parsing Ascendente



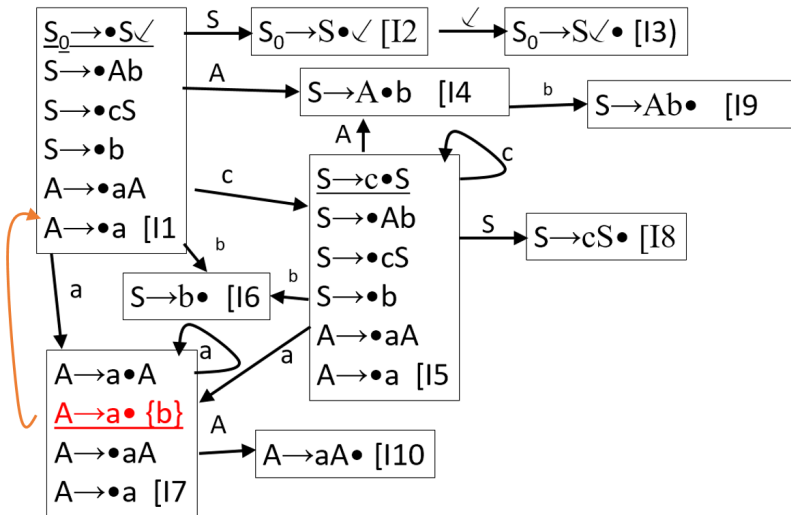
La grammatica è LR(0)?

- No, la grammatica NON è LR(0)
- Passiamo alla tecnica LALR(1): calcoliamo i look-ahead set per le candidate di riduzione

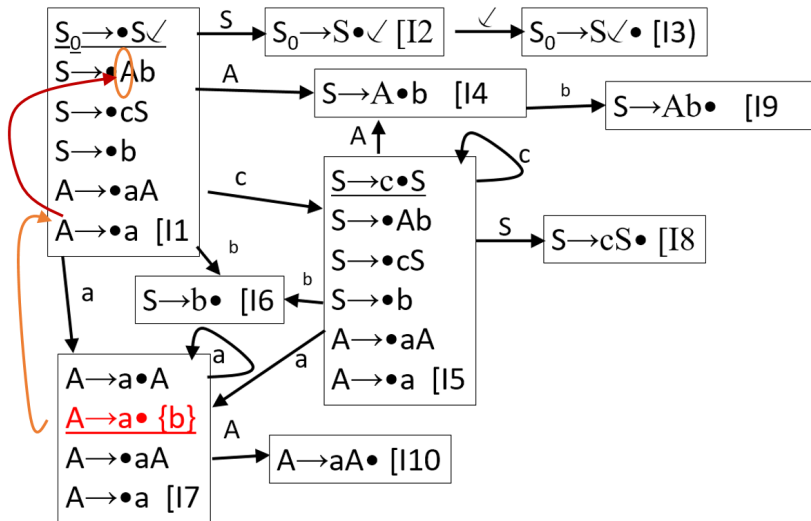
Parsing Ascendente



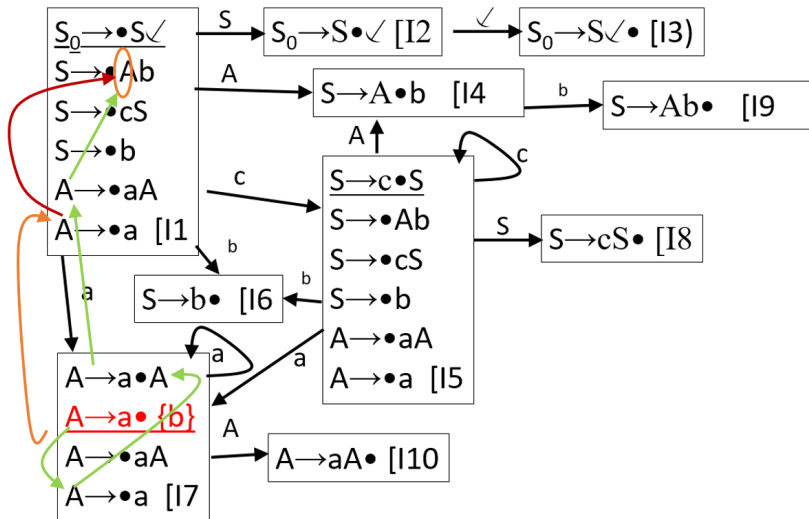
Parsing Ascendente



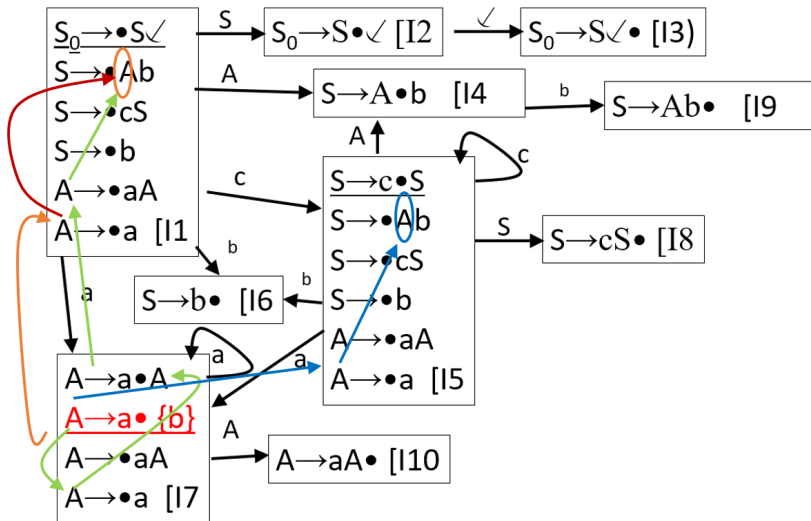
Parsing Ascendente



Parsing Ascendente



Parsing Ascendente

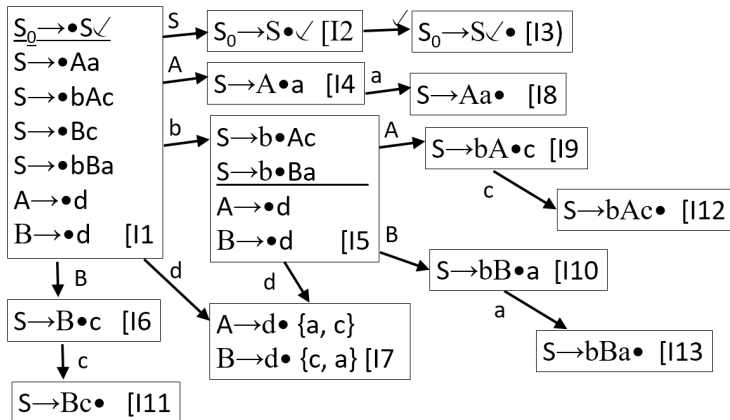


Superare i Limiti di LALR(1)

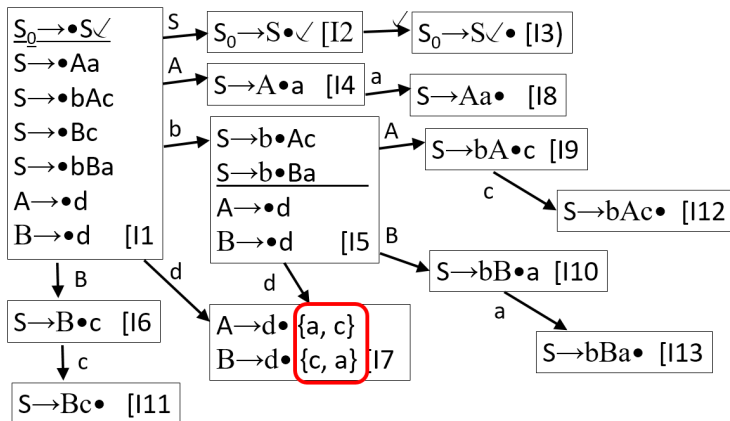
Consideriamo la grammatica seguente

- $S_0 \rightarrow S$ ✓
 $S \rightarrow A a$
 $S \rightarrow b A c$
 $S \rightarrow B c$
 $S \rightarrow b B a$
 $A \rightarrow d$
 $B \rightarrow d$

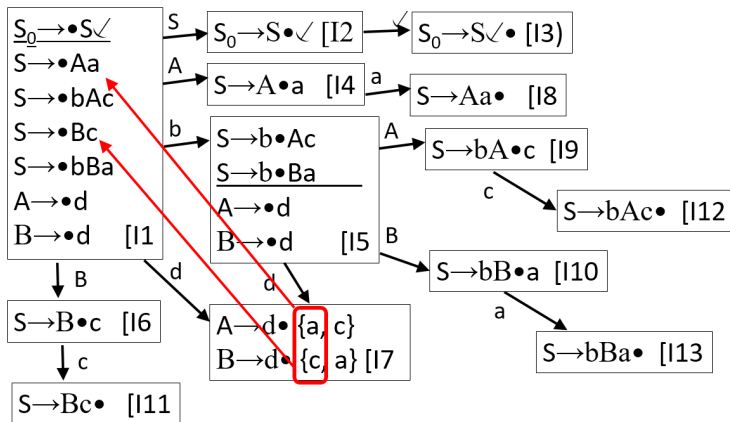
Parsing Ascendente



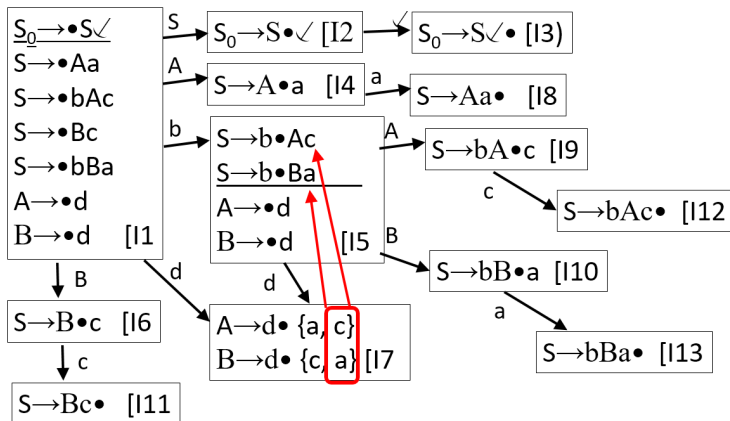
Parsing Ascendente



Parsing Ascendente



Parsing Ascendente



Automa LR(1)

- È la versione più potente del parsing ascendente con prospezione 1
- Cerca di separare la provenienza dei look-ahead, per gestire separatamente i contesti dei sotto-alberi
- Come? Calcolando i look-ahead di ogni candidata fin dall'inizio.
- Ogni candidata ha **SEMPRE** associato un Look-ahead set.

Creazione Automa LR(1)

- Nello stato I_1 ,
la candidata core $S_0 \rightarrow \bullet S \searrow$, ha
 $LA(I_1, S_0 \rightarrow \bullet S \searrow) = \emptyset$
- In ogni stato s , nelle candidate di completamento
 $N \rightarrow \bullet \beta$, e le candidate $c_j : M_j \rightarrow \alpha_j \bullet N \beta_j$
dove $Seg(N, c_j) = Ini(\beta_j)$ se $\neg Annullabile(\beta_j)$ o
 $Seg(N, c_j) = Ini(\beta_j) \cup LA(s, M_j \rightarrow \alpha_j \bullet N \beta_j)$
 $LA(s, N \rightarrow \bullet \beta) = \cup_{c_j} Seg(N, c_j)$

Creazione Automa LR(1)

- Le candidate core di uno stato diverso da I_1 mantengono il look-ahead set delle candidate di provenienza
- Quindi, se esiste $s \xrightarrow{X} \bar{s}$, (con $X \in (\Sigma \cup V)$),
abbiamo $N \rightarrow \alpha \bullet X \beta \in s$ e $N \rightarrow \alpha X \bullet \beta \in \bar{s}$
con $LA(\bar{s}, N \rightarrow \alpha X \bullet \beta) = LA(s, N \rightarrow \alpha \bullet X \beta)$

Creazione Automa LR(1)

- I look-ahead set contribuiscono anche all'identità dello stato, quindi
 - se lo spostamento genera uno stato con le stesse candidate core di uno stato già esistente, ma con look-ahead set diversi,
 - si crea un nuovo stato

Condizioni LR(1)

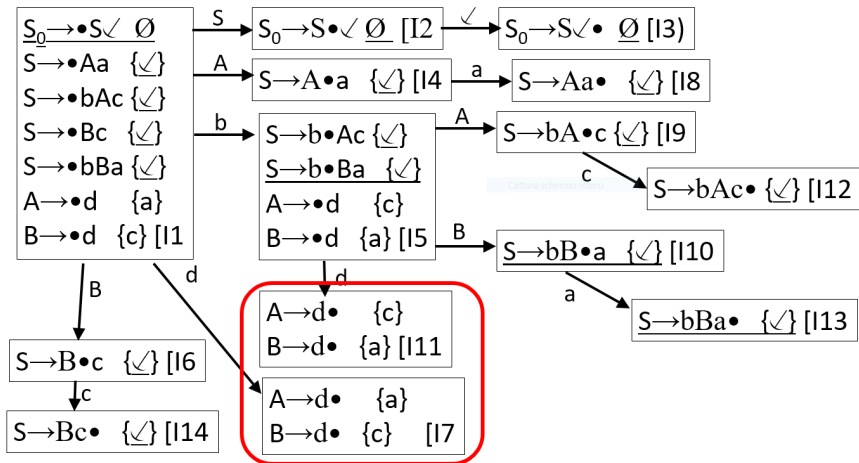
- Dato uno stato che presenta conflitti spostamento/riduzione o riduzione/riduzione,
- Le condizioni sono le stesse del caso LALR(1), cioè
 - i look-ahead set delle candidate di riduzione devono essere disgiunti tra di loro
 - e devono essere disgiunti dalle mosse uscenti

Automa LR(1)

Riconsideriamo la grammatica seguente

- $S_0 \rightarrow S$ ✓
 $S \rightarrow A a$
 $S \rightarrow b A c$
 $S \rightarrow B c$
 $S \rightarrow b B a$
 $A \rightarrow d$
 $B \rightarrow d$

Parsing Ascendente



Parsing Ascendente

